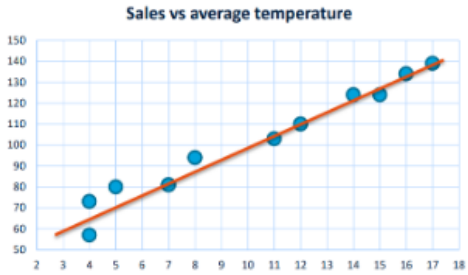
Lecture 7 - Linear Regression

Supervised Learning: the correct answer are given during training
Unsupervised Learning: no answers given, try to find patterns in the data

Month	Average Temp. (°C)	Ice Cream sales (Kg)
January	4	73
February	4	57
March	7	81
April	8	94
May	12	110
June	15	124
July	16	134
August	17	139
September	14	124
October	11	103
November	7	81
December	5	80



Multiple features (variables).

Size (feet²) x_1	Number of bedrooms x_2	Number of floors x_3	Age of home (years) x_4	Price ($1000) y
2104	5	1	45	460
1416	3	2	40	232
1534	3	2	30	315
852	2	1	36	178
...	...	...	...	...

m = number of training samples

Notation

m: number of training instances.

x: input variables

y: output variables

(x, y): one training instance

(x^i, y^i): the i-th training instance

$x_j^{(i)}$: value of feature j in the i-th training instance

h: hypothesis that maps x to y

θ: parameters of the model

θ_0: **bias term** (or intercept term)

One variable (**Univariate** linear regression):

$$y = h_\theta(x) = \theta_0 + \theta_1 x$$

Multiple variables / features (**Multivariate** linear regression):

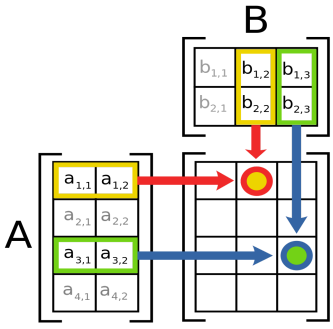
$$y = h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \ldots + \theta_n x_n = \theta^T x$$

Here θ^T and x are vectors containing all values. Note that for this to work x_0 is always defined as 1 (bias term).

Cost function

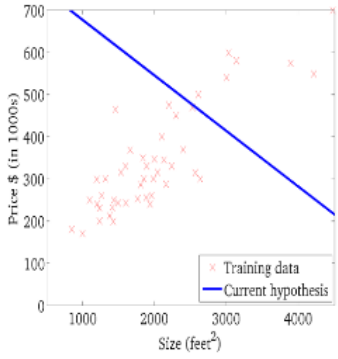
Minimize:

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2$$



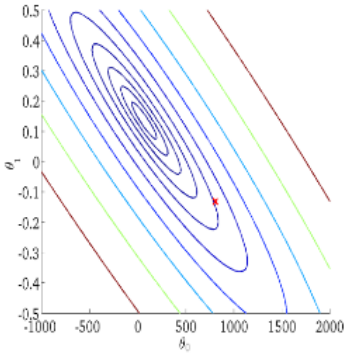
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)



Solving the cost function

We can rewrite the cost function to $(X\theta - y)^T(X\theta - y)$. We can minimize this by taking the derivate and setting it to 0. Solving for θ gives $\theta = (X^T X)^{-1}(X^T y)$. This is very slow for handling data sets with lots of features since it needs to do a matrix inversion. So minimizing θ has to be done using another optimization technique.

SSE cost function is always **convex**. i.e. the line segment between any two points always lies above the graph.

Gradient Descent

Finds local minima or maxima. Repeat until convergence:

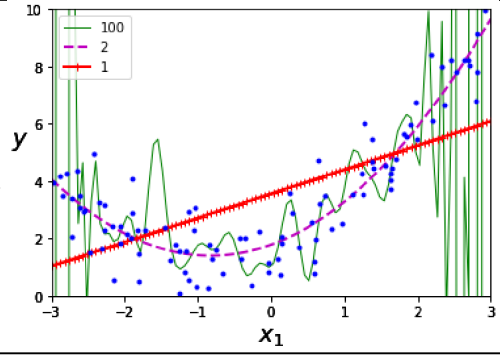
$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

For univariate linear regresion you only do this for $j = 0$ and $j = 1$.
Reminder that for $j = 0$ the last $x_0^{(i)}$ term is always 1 (bias term).

The α in the first formula stands for the **learning parameter**. If α is too small, the algorithm will converge too slowly. If α is too big, the algorithm may overshoot and not converge at all.

Overfitting:

A high number of features means it is very likely to overfit. Manually select which features to keep and which not

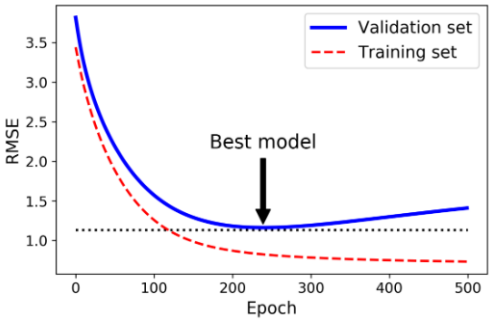


Also shows polynomial formulas fall under linear regression

Feature Scaling: Standardize the range of features (preferably the [-1, 1] range). Optimization algorithms do not think for you!

Regularization: extra parameter that balances generalization and fitting data well. A very effective tool against overfitting.

Cost Function: $J(\theta) = \frac{1}{2m}(\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^{n} \theta_j^2)$

Gradient Descent: $\theta_j := \theta_j(1 - \alpha\frac{\lambda}{m}) - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}$

Learning Curvers: Can cut off learning after minimum has been reached